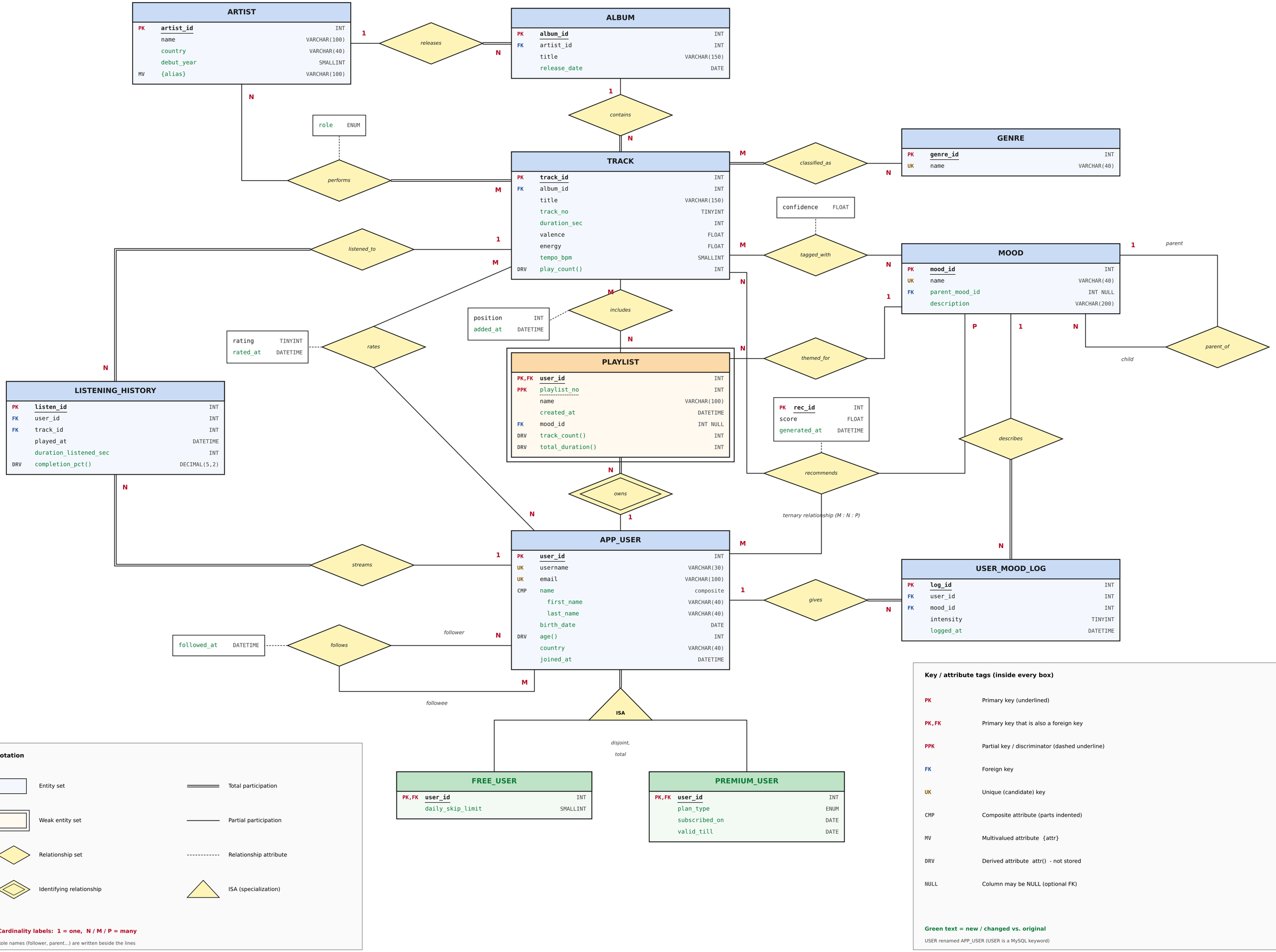
Music Mood Recommendation Database — Enhanced ER Diagram (v2)

DBMS Final Project · Phase 1 · Korth-style notation · every key is tagged with its type · green text = added/changed vs. the original diagram



Notation

- Entity set
- Weak entity set
- Relationship set
- Identifying relationship
- Total participation
- Partial participation
- Relationship attribute
- ISA (specialization)

Cardinality labels: 1 = one, N / M / P = many

Role names (follower, parent…) are written beside the lines

Key / attribute tags (inside every box)

PK	Primary key (underlined)
PK,FK	Primary key that is also a foreign key
PPK	Partial key / discriminator (dashed underline)
FK	Foreign key
UK	Unique (candidate) key
CMP	Composite attribute (parts indented)
MV	Multivalued attribute {attr}
DRV	Derived attribute attr() - not stored
NULL	Column may be NULL (optional FK)

Green text = new / changed vs. original

USER renamed APP_USER (USER is a MySQL keyword)

Design notes for the v2 ER diagram

Page 1 is the diagram itself. These pages explain what changed from the original and why, so you can defend every choice in the viva.

1. What changed from the original diagram

Change	Why it is there
Your 3 suggestions added: **logged_at** (USER_MOOD_LOG), **duration_listened_sec** (LISTENING_HISTORY), **generated_at** (RECOMMENDATION)	Time-based analysis (mood trends, skip rate, fresh vs. stale recommendations).
USER renamed APP_USER	USER is a keyword/function name in MySQL and invites quoting problems. Every user_id FK stays the same.
Specialization (ISA): APP_USER into FREE_USER and PREMIUM_USER (disjoint, total)	Gives the class topic of generalization/specialization a real home, with subtype-only attributes.
Weak entity: PLAYLIST depends on APP_USER through the identifying relationship *owns*; key = (user_id, playlist_no)	A playlist number only makes sense inside one user. Shows weak entity, discriminator and a composite FK from PLAYLIST_TRACK.
Recursive relationships: MOOD parent_of MOOD (mood hierarchy) and APP_USER follows APP_USER	Self-referencing FKs, role names, and room for recursive queries later.
Ternary relationship *recommends* (APP_USER, TRACK, MOOD) drawn as one diamond	In the old diagram it was just an entity with three FKs. It is really a 3-way relationship, and the diagram now says so (still mapped to a table with surrogate key rec_id).
Attribute kinds: composite (name), multivalued ({alias}), derived (age(), play_count(), completion_pct(), track_count(), total_duration())	Each kind maps to the relational model differently, so you can explain the mapping rules.
Participation constraints (double lines) and cardinalities on every link	The original had neither; the guidelines ask for all relations and cardinalities.
More descriptive attributes: country, birth_date, joined_at, release_date, duration_sec, tempo_bpm, track_no, role, rated_at, added_at, followed_at, etc.	Makes sample data and queries more realistic (age groups, long vs. short tracks, newest releases).

2. Relationships and cardinalities

Relationship	Between	Cardinality	Participation (double line)	Attributes
releases	ARTIST - ALBUM	1 : N	ALBUM total	-
contains	ALBUM - TRACK	1 : N	TRACK total	-
performs	ARTIST - TRACK	M : N	TRACK total	role
classified_as	TRACK - GENRE	M : N	TRACK total	-
tagged_with	TRACK - MOOD	M : N	partial both	confidence
rates	APP_USER - TRACK	M : N	partial both	rating, rated_at
follows (recursive)	APP_USER - APP_USER	M : N	partial both	followed_at
parent_of (recursive)	MOOD - MOOD	1 : N	partial both	-
gives	APP_USER - USER_MOOD_LOG	1 : N	LOG total	-
describes	MOOD - USER_MOOD_LOG	1 : N	LOG total	-
streams	APP_USER - LISTENING_HISTORY	1 : N	LISTENING_HISTORY total	-
listened_to	TRACK - LISTENING_HISTORY	1 : N	LISTENING_HISTORY total	-
owns (identifying)	APP_USER - PLAYLIST	1 : N	PLAYLIST total (weak)	-
includes	PLAYLIST - TRACK	M : N	partial both	position, added_at
themed_for	MOOD - PLAYLIST	1 : N	partial both	-
recommends (ternary)	APP_USER - TRACK - MOOD	M : N : P	partial all	rec_id, score, generated_at
ISA	APP_USER - FREE_USER / PREMIUM_USER	disjoint	total	plan_type, valid_till, ...

3. Where the class topics show up (Korth, ch. 1-7)

I assumed ch. 1-7 means: intro, relational model, basic / intermediate / advanced SQL, ER model, relational database design. If your slides are numbered differently, tell me and I will re-map this table.

Topic	Where this design uses it
Relational model, keys, referential integrity	Primary keys, composite keys (RATING, TRACK_ARTIST, PLAYLIST_TRACK), candidate keys (UK), foreign keys with ON DELETE / ON UPDATE rules, a composite FK (playlist_track to playlist).
Basic and intermediate SQL: joins, aggregates, GROUP BY / HAVING, subqueries, set operations, views	Top tracks per mood, average rating per genre, users with no listens (LEFT JOIN), UNION/INTERSECT of tracks for two moods, views such as v_track_popularity.
Integrity constraints	CHECK (rating 1-5, intensity 1-5, valence/energy/confidence/score in 0..1, follower <> followee, valid_till >= subscribed_on), NOT NULL, UNIQUE, DEFAULT.
Advanced SQL: functions, procedures, triggers, recursion, window functions	Derived attributes as functions/views (age, play_count, completion_pct); trigger to keep a log or validate data; recursive CTE over the mood hierarchy and the follow graph; RANK() of tracks inside a mood.
ER model: weak entity, specialization, ternary, recursive, composite / multivalued / derived attributes, participation	All present on page 1 (see section 1).
Relational design: functional dependencies, 2NF / 3NF / BCNF, lossless decomposition	Next step. The composite-key tables give natural partial-dependency examples, and the mapping below is the starting schema for the normalization report.

4. ER to relational mapping (19 tables)

Derived attributes (age, play_count, completion_pct, track_count, total_duration) are **not** stored. The composite attribute *name* is flattened into first_name and last_name. The multivalued {alias} becomes its own table.

Table	Columns (PK underlined by position: listed first / marked PK)	Foreign keys
app_user	**PK user_id**, username UK, email UK, first_name, last_name, birth_date, country, joined_at	-
free_user	**PK/FK user_id**, daily_skip_limit	user_id -> app_user
premium_user	**PK/FK user_id**, plan_type, subscribed_on, valid_till	user_id -> app_user
artist	**PK artist_id**, name, country, debut_year	-
artist_alias	**PK/FK artist_id**, **PK alias**	artist_id -> artist
album	**PK album_id**, artist_id, title, release_date	artist_id -> artist
track	**PK track_id**, album_id, title, track_no, duration_sec, valence, energy, tempo_bpm (UK album_id + track_no)	album_id -> album
genre	**PK genre_id**, name UK	-
mood	**PK mood_id**, name UK, parent_mood_id (NULL), description	parent_mood_id -> mood
track_artist	**PK/FK track_id**, **PK/FK artist_id**, role	track, artist
track_genre	**PK/FK track_id**, **PK/FK genre_id**	track, genre
track_mood	**PK/FK track_id**, **PK/FK mood_id**, confidence	track, mood
user_mood_log	**PK log_id**, user_id, mood_id, intensity, logged_at	app_user, mood
listening_history	**PK listen_id**, user_id, track_id, played_at, duration_listened_sec	app_user, track
rating	**PK/FK user_id**, **PK/FK track_id**, rating, rated_at	app_user, track
user_follow	**PK/FK follower_id**, **PK/FK followee_id**, followed_at	both -> app_user
playlist	**PK/FK user_id**, **PK playlist_no**, name, created_at, mood_id (NULL)	user_id -> app_user, mood_id -> mood
playlist_track	**PK** (user_id, playlist_no, track_id), position, added_at	(user_id, playlist_no) -> playlist, track_id -> track
recommendation	**PK rec_id**, user_id, track_id, mood_id, score, generated_at	app_user, track, mood

Modeling choice to know about: a track belongs to exactly one album (singles count as a one-track album). That keeps track.album_id a plain NOT NULL FK.